AIVV: Neuro-Symbolic LLM Agent-Integrated Verification and Validation for Trustworthy Autonomous Systems

Jiyong Kwon[1] **Ujin Jeon**[2] **Sooji Lee**[3] **Guang Lin**[1,4]

[1]School of Mechanical Engineering, Purdue University
[2]School of Electrical and Computer Engineering, Purdue University
[3]Department of Computer Science, Purdue University
[4]Department of Mathematics, Purdue University
West Lafayette, IN 47907, USA
{kwon165, ujeon, lee5264, guanglin}@purdue.edu

arXiv:2604.02478v2 [cs.AI] 7 Aug 2026

Abstract

Deep learning models excel at detecting anomaly patterns in normal data. However, they do not provide a direct solution for anomaly classification and scalability across diverse control systems, frequently failing to distinguish genuine faults from nuisance faults caused by noise or the control system's large transient response. Consequently, because algorithmic fault validation remains unscalable, full Verification and Validation (V&V) operations are still managed by Human-in-the-Loop (HITL) analysis, resulting in an unsustainable manual workload. To automate this essential oversight, we propose Agent-Integrated Verification and Validation (AIVV), a hybrid framework that deploys Large Language Models (LLMs) as a deliberative outer loop. Because rigorous system verification strictly depends on accurate validation, AIVV escalates mathematically flagged anomalies to a role-specialized LLM council. The council agents perform collaborative validation by semantically validating nuisance and true failures based on natural-language (NL) requirements to secure a high-fidelity system-verification baseline. Building on this foundation, the council then performs system verification by assessing post-fault responses against NL operational tolerances, ultimately generating actionable V&V artifacts, such as gain-tuning proposals. Experiments on a time-series simulator for Unmanned Underwater Vehicles (UUVs) demonstrate that AIVV successfully digitizes the HITL V&V process, overcoming the limitations of rule-based fault classification and offering a scalable blueprint for LLM-mediated oversight in time-series data domains.

1 Introduction

In mission-critical domains such as Unmanned Underwater Vehicles (UUVs), automated anomaly detection systems must process telemetry that is noisy, highly stochastic, and sparse in genuine fault events (Pang et al., 2021). Contemporary research relies heavily on deep learning architectures (e.g., RNNs, Transformers) coupled with robust uncertainty quantification to establish rigorous residual bounds and control the marginal error rate (Hundman et al., 2018; Tuli et al., 2022; Angelopoulos & Bates, 2021). While these mathematical models are highly efficient computationally and excel at detecting extreme, catastrophic failures, they do not provide a direct solution for fault classification in system redesign and development. Consequently, they frequently fail to distinguish genuine faults from nuisance faults caused by nominal environmental noise or the control system's large transient responses. Although Jeon et al. (2025) proposed a solution to ensure proper coverage in fault detection and identification, the proposed algorithm does not demonstrate clear scalability across diverse systems. To detect anomalies such as gradual sensor drift, mathematical safety bounds must be tightened, which inevitably triggers a cascade of false-positive alarms.

Furthermore, these purely mathematical systems are limited to binary detection; they cannot assess downstream system impact or propose corrective control re-designs.

Because algorithmic fault validation remains unscalable, managing these false-positive anomalies and performing rigorous Verification and Validation (V&V) has heavily relied on the Human-in-the-Loop (HITL) paradigm, in which human domain experts provide the semantic reasoning (Zanzotto, 2019). However, human operators introduce significant latency and cannot scale to monitor thousands of sensor streams simultaneously (Cummings, 2014). While a mathematical sensor flags a fault in milliseconds, a human engineering team may require several minutes to hours to manually triage the alert, analyze the telemetry, and safely validate a controller retuning to verify the system. This results in an unsustainable manual workload, highlighting the urgent need to automate this essential oversight.

To automate and reduce the delay of this oversight, Large Language Models (LLMs) offer a compelling alternative for rapid semantic reasoning. The industry increasingly seeks the reasoning capabilities of LLMs to interpret complex physical environments; however, their deployment in mission-critical automated systems remains a formidable systems engineering challenge. While LLMs excel at zero-shot diagnosis, their propensity for hallucination and lack of deterministic mathematical rigor preclude their direct integration into reliable, real-world applications (Ji et al., 2023). Hypersensitivity to short-lived environmental fluctuations without strict mathematical bounding can lead to unwarranted mission aborts or costly vehicle retrieval operations, ultimately eroding operator trust (Van de Merwe et al., 2024). To mitigate the hallucinations of standalone models (Du et al., 2023), recent research has leveraged LLM-based Multi-Agent Systems (MAS) (Xi et al., 2023) to digitize the collaborative cross-validation process of a human engineering team. Yet, placing a MAS directly in the continuous-time inference loop would introduce prohibitive latency due to the sequential nature of LLM API calls and generation times.

To bridge the gap between mathematical rigor and semantic reasoning, and to safely automate the HITL process, we propose Agent-Integrated Verification and Validation (AIVV). It operates as a two-layer hybrid neuro-symbolic architecture that structurally bifurcates the workload: a high-speed mathematical frontline rapidly flags potential anomalies, escalating mathematically verified limit breaches to a deliberative outer loop managed by a role-specialized LLM council. Because rigorous system verification depends on accurate validation, the council agents perform collaborative validation by semantically validating nuisance and true failures against natural-language (NL) requirements to establish a high-fidelity system verification baseline. Built on this foundation, the council then performs system verification by assessing post-fault responses against natural-language operational tolerances, thereby generating actionable V&V artifacts, such as gain-tuning proposals.

Contributions. 1) Role-based LLM agents automate V&V processes, resolving fault diagnoses via collaborative validation against natural-language operational tolerances and a majority voting system to filter false alarms. **2)** We introduce a neuro-symbolic gating mechanism that couples an MC Dropout LSTM and conformal prediction with an LLM council, ensuring mathematically flagged anomalies are semantically validated as either nuisance or true failures with natural-language requirements. **3)** AIVV translates anomaly flags into structured engineering artifacts (e.g., gain-tuning proposals), and ensures safe online adaptation by executing candidate model updates on a temporarily cloned engine before live promotion.

2 Related Work

LLM-Based MAS. LLM-based multi-agent systems (MAS) enable groups of intelligent agents to coordinate on complex reasoning and intensive workloads (Chen et al., 2025; Tran et al., 2025). In the context of mission-critical verification, a MAS offers significant robustness advantages over a monolithic LLM, noting that standalone models act as a single point of failure and remain highly susceptible to hallucinations (Du et al., 2023).

HITL and V&V. The paradigm of operators acting within a closed control loop is deeply established via Human-in-the-Loop (HITL) frameworks (Licklider, 1960; Zanzotto, 2019).

Within these environments, Verification and Validation (V&V) is an active process that requires analyzing telemetry to safely validate system responses and provide feedback on system failure modes. However, Cummings (2014) highlights that as the volume of high-frequency sensor streams grows, manual HITL oversight introduces significant latency and becomes a workload bottleneck. In contemporary agentic AI and runtime verification research, V&V is typically treated as a static, post hoc procedure that relies on offline verification (Koohestani, 2025), rendering standard LLM evaluation methods unsuitable for continuous operational tasks. Unlike (Srinivasan et al., 2025), which focuses on broad simulator-level dependability assurance, AIVV focuses on mathematical gating and requirements-grounded engineering V&V for high-frequency control-system telemetry, including safeguarded adaptation and a controller gain-tuning proposal for system design.

Mathematical Paradigms in LSTM-Based Anomaly Detection. Data-driven Long Short-Term Memory (LSTM) architectures are standard for anomaly detection in continuous control systems, mapping complex temporal dependencies without explicit physical models (Malhotra et al., 2015). Recent hybridized frameworks improve robustness to corrupted telemetry, utilizing mechanisms such as BiLSTM-Attention for Autonomous Underwater Vehicles (Dong & Huo, 2025) and channel-spatial attention for industrial manufacturing (Lobo et al., 2026). To mitigate overconfident predictions, recent approaches integrate Uncertainty Quantification (UQ), such as LSTMAE-UQ, which uses Monte Carlo (MC) Dropout to approximate Bayesian inference and capture epistemic uncertainty (Wiessner et al., 2024; Gal & Ghahramani, 2016). Similarly, a redundancy-based algorithm for accurate fault detection was introduced; However, LSTM-based fault detection frameworks still face scalability limitations and often require domain-specific designs to achieve accurate fault classification (Jeon et al., 2025). To translate this uncertainty into strict operational bounds, Conformal Prediction (CP) is frequently applied to provide finite-sample marginal coverage guarantees (Angelopoulos & Bates, 2021). However, in dynamic environments, sudden shifts in distribution during transient maneuvers violate CP's strict exchangeability assumption, leading to false alarms (Barber et al., 2023). Consequently, while these mathematically bounded autoencoders excel at identifying statistical outliers (Mbakob Ndagoua & Hasnaoui, 2025), they suffer from critical limitations. Furthermore, their inherent semantic blindness prevents them from reliably differentiating genuine faults from mathematically identical transient responses caused by intentional maneuvers. This inability to scale and contextually filter nuisance faults establishes the strict requirement for the higher-level, reasoning-based validation introduced by our framework.

3 Methodology

The AIVV framework is designed for online failure validation and system response verification on dynamic UUV yaw angle time-series data under environmental noise. Each incoming sample is processed as a sliding-window sequence through a mathematical engine and a highly structured, role-based Multi-Agent System (MAS) powered by LLMs.

The architecture is explicitly designed to ground language model reasoning in statistical reality. The Mathematical Engine Layer provides the computational core by employing an LSTM utilizing Monte Carlo dropout for Bayesian approximation to produce a point prediction and an epistemic uncertainty estimate, while a conformal prediction maintains a statistically guaranteed conformal bound C_α.

When this mathematical bound is breached, the decision logic and adaptation strategy are controlled by MAS. For rigorous, hallucination-resistant execution, the agent pipeline is divided into three sequential phases: deterministic gating, deliberative adjudication, and adaptive fine-tuning with safeguards. This ensures that normal operational fluctuations are resolved at the deterministic level, while detected fault cases are escalated to the council for investigation and a vote on the underlying system-level anomaly.

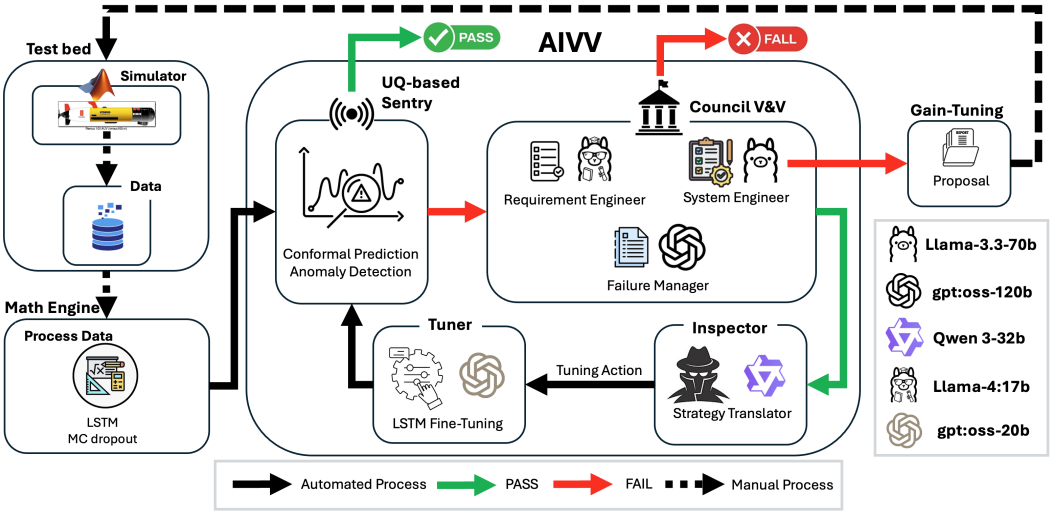


Figure 1: AIVV framework, illustrating the sequential flow of the system.

3.1 Mathematical Engine Layer

This Layer provides mathematical proposals for predictive uncertainty that deliver a statistical context for downstream LLMs. It is implemented as an MC Dropout LSTM f_θ, which enables uncertainty estimation at inference time. For a given input sample x_t, the model performs stochastic forward passes to compute a predictive mean $\hat{y}_t$ and standard deviation σ_t.

To establish a strict confidence interval, we integrate a conformal prediction. Using a held-out calibration set, the predictor computes a conformal bound C_α that guarantees a user-defined coverage rate $1 - \alpha$. The algorithm for the predictive mean $\hat{y}_t$, uncertainty quantification, and the offline conformal calibration procedure are detailed in Appendix A.

3.2 The Multi-Agent System Pipeline

The multi-agent escalation path serves as a true failure validation and trustworthy verification for system development. The pipeline operates in three distinct phases:

3.2.1 *Phase 1: Deterministic Conformal Gating (The Sentry)*

The Sentry serves as a deterministic, non-LLM hard gate that bridges the mathematical engine and the Council. It evaluates the absolute residual e_t sample-by-sample:

$$e_t = |\hat{y}_t - y_t| \tag{1}$$

against the calibrated conformal bound C_α. Its decision rule d_t is:

$$d_t = \begin{cases} \texttt{FAIL}, & \text{if } e_t > C_\alpha, \\ \texttt{PASS}, & \text{otherwise.} \end{cases} \tag{2}$$

A sample is forwarded to the LLM council agents only when this conformal bound is violated. This ensures that the bounds are tightened to detect even gradual failure before deliberative reasoning is invoked, thereby preserving computational efficiency.

3.2.2 *Phase 2: Deliberative Adjudication (The Council)*

When the Sentry flags a sample as anomalous ($d_t = \texttt{FAIL}$), it is escalated directly to the Council. Three independent LLM-based agents evaluate the statistical context and cast one equal vote (`PASS` or `FAIL`). The system applies a 2-of-3 majority rule: if at least two agents

Algorithm 1 AIVV Pipeline

Require: Stream sample x_t, base engine f_θ, significance level α
Ensure: Final label $d_t \in \{\texttt{PASS}, \texttt{FAIL}\}$ and updated engine state (if promotion occurs)

for each sample x_t **do**
  $d_t \leftarrow$ **Sentry**(x_t, f_θ, α)
  if $d_t = \texttt{FAIL}$ **then**
    for $k = 1$ to 2 **do**
      if MajorityVote(**Council**(context)) $= \texttt{FAIL}$ **then**
        $d_t \leftarrow \texttt{FAIL}$; **break**
      end if
      if $k = 2$ **then**
        $d_t \leftarrow \texttt{PASS}$; **break**
      end if
      Tuning Action: $a_t \leftarrow$ **Inspector**(context)
      $f_\theta^{tmp} \leftarrow \text{clone}(f_\theta)$
      $f_\theta^{tmp} \leftarrow$ **Tuner**(a_t, f_θ^{tmp})
      if Sentry$(x_t, f_\theta^{tmp}, \alpha) = \texttt{PASS}$ **then**
        promote $f_\theta \leftarrow f_\theta^{tmp}$
        $d_t \leftarrow \texttt{PASS}$; **break**
      else
        keep f_θ; discard f_θ^{tmp} {Loop to $k = 2$ for final deliberation}
      end if
    end for
  end if
  output d_t
end for

vote FAIL, the sample is confirmed as a true failure. If at least two votes are PASS, the sample is deemed an abrupt maneuver, and the pipeline proceeds to Phase 3 for system adaptation.

- **Requirements Engineer:** An LLM-based council member (LLaMA-4-17B) that evaluates whether the current system behavior satisfies operational requirements at the specific sample flagged by the mathematical model. Given the system's sensor telemetry data, training constraints, and a natural-language-focused requirements set, it returns a compliance vote, the cited requirement section, and an analytical justification for any identified violations.

- **Failure Manager:** An LLM-based council member (GPT-OSS-120B) designed to evaluate the system in failure mode and effect analysis with given failure management requirements. By monitoring its evaluation in the recent telemetry window, this agent assesses the severity of the trajectory deviation. It returns a compliance vote, a categorical failure-mode level, and a requirement-grounded rationale.

- **System Engineer:** An LLM-based council member (LLaMA-3.3-70B) possessing explicit domain knowledge of both the UUV dynamics (e.g., Nomoto model parameters, PID controller parameters) and the mathematical detection mechanism (e.g., MC dropout uncertainty, conformal boundary calibration). This agent returns a compliance vote from a technical perspective, based on the vulnerability of the pure mathematical model. Evaluating these metrics distinguishes genuine failures from FPR induced by environmental noise and dynamic maneuvering. Furthermore, upon confirming a true fault, the agent conducts verification and validation (V&V) and generates a control-system gain-tuning proposal conditioned on the failure-mode operation to stabilize the system.

3.2.3 *Phase 3: The Adaptation Pipeline (Inspector & Tuner)*

When the Councils decide that an anomaly flagged by the Sentry is actually a nuisance failure, the system uses this as a fine-tuning trigger to retrain/recalibrate the math engine.

- **Inspector Agent:** An LLM-based strategy translator (`Qwen3-32B`). It receives the current residual, conformal bound, and the Council's vote summaries (including reasoning and confidence). It synthesizes this to output a discrete tuning action: `RECALIBRATE` (suggesting a new conformal significance level α), `FINE_TUNE` (suggesting network training hyperparameters), or `TRY_BOTH`.
- **Tuner Agent:** An LLM-based executor (`GPT-OSS-20B`) that applies the Inspector's selected action to a temporary cloned engine rather than the deployed model. It returns a reevaluated candidate state (updated prediction, bound, and error) along with a detailed log of the model's reasoning and key statistics.

To close the loop, the Sentry (Phase 1) is invoked a second time. It tests the Tuner's candidate model against the updated conformal bound. If the candidate resolves the conformal violation, the temporary clone is promoted to the new updated model. If it fails, the deployed engine remains unchanged to prevent degradation of the mathematical model and protect against catastrophic forgetting. The Council is then re-invoked for a second deliberation round: a majority `FAIL` confirms a true fault, while a majority `PASS` overrides the Sentry flag and labels the sample as nominal. Detailed prompts and representative JSON execution traces for each LLM agent are provided in Appendices C and D, respectively.

3.3 V&V Feedback and Corrective-Action Interface

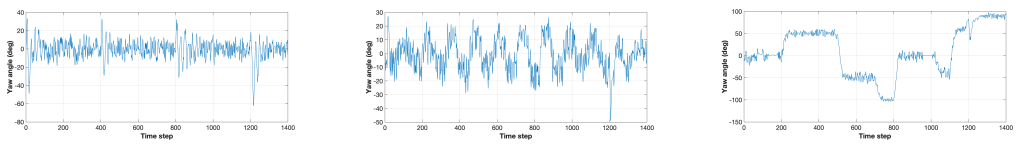
This framework introduces a comprehensive verification and validation (V&V) loop for system-level design and testing. During this phase, the council agents assume the roles of system V&V engineers, operating based on specified system requirements, failure management protocols, and system descriptions. Each agent continuously monitors the predicted sensor output to determine whether the system's trajectories satisfy the prescribed requirements under both normal operating conditions and failure modes.

Following the monitoring phase, each agent evaluates the test results. The requirements engineer specifically monitors whether the predicted trajectory falls within the normal operating range for the given system requirements. The failure manager strictly diagnoses the failure modes of the predicted output data. The System Engineer agent then aggregates the findings from the Requirement Engineer and the Failure Manager and transitions into gain-tuning mode. Based on the specified system engineering requirements, this agent proposes adjusted gain-tuning parameters for the UUV control system. Details are provided in Appendix I.

4 Experimental Setup

4.1 Dataset Generation

We evaluated the AIVV using a Simulink model (Cybergalactic, 2023) of the REMUS 100 UUV with an IMU sensor fusion model provided by MathWorks (The MathWorks, Inc., 2023). To evaluate the robustness and scalability of the proposed framework under a



(a) Dataset 1. (b) Dataset 2. (c) Dataset 3.

Figure 2: (a) Hovering, (b) Lawnmower Mapping Pattern, and (c) Complex Mission.

challenging environment, we added sensor-drift bias and environmental non-Gaussian noise, and generated three maneuver cases (Dataset 1: Hovering; Dataset 2: Lawnmower Mapping Pattern; Dataset 3: Complex Mission). Details of the datasets are provided in Appendix E.

For these experiments, the yaw angle data were sampled at 20 ms intervals, and the initial 1,400 raw time-series data points were split in a 70/30 ratio. Generating sequence-target pairs from these splits with a sliding window size of $W = 10$ and a prediction horizon of $H = 2$ yielded exactly 969 training phase and 409 independent testing phase.

4.2 Failure Mode Generation

To simulate the yaw-angle failure scenario of the REMUS 100 in various maneuvers, we injected failures into the IMU sensor during the test phase. At 1200 time step, we injected an electrical sensor failure into Datasets 1 and 2 and a mechanical damper failure into Dataset 3. As shown in Figures 2(a) and 2(b), high-spike faults occur exactly at 1200 time step, but the REMUS 100 control system stabilizes the yaw-angle response to the normal operating level. However, in Figure 2(c), because the mechanical damper failure cannot be stabilized by the inner control system, the yaw angle shows a hardover failure response after 1200 time step.

4.3 Adaptive Engine Configuration

To ensure the mathematical engine maintains appropriately calibrated sensitivity, the hyperparameters are dynamically changed by the adaptation pipeline rather than remaining static. While the baseline conformal prediction threshold is initialized at $\alpha = 0.05$, when the Tuner agent is called, it adjusts this significance level within a bounded range of $[0.01, 0.10]$ (between 90% to 99% confidence). This dynamic bounding allows the system to effectively adapt to even dramatic transient responses and unexpected environmental noise, and to prevent nuisance faults from overwhelming the multi-agent reasoning loop. Details of hyperparameters are provided in Appendix B.

When the Council overrides a mathematical Sentry flag by voting `PASS` on a nuisance fault, the inspector agent performs fine-tuning or recalibration and dynamically formulates the strategy based on the council's JSON log. The Inspector generates a conservative adaptation strategy within an epoch range of 50 to 200 epochs, using a learning rate between 0.00001 and 0.001. This adaptive epoch scaling allows the model to incorporate an updated mathematical model following a confirmed structural change without catastrophic forgetting previous normal behaviors.

5 Experiment Results

This section presents the empirical evaluation of the AIVV framework across the three maneuver cases. First, we evaluate the system's ability to validate true versus nuisance failures and analyze the resulting efficiency of the adaptation pipeline. Next, we present a structural ablation study to demonstrate the importance of each layer. Finally, we discuss our role-specific model alignment.

All the experiments were conducted in Intel Data Center GPU Max Series accelerators, and for LLM inference, we used Groq's custom Language Processing Units (LPUs). For further hardware details, see Appendix H.

5.1 Validation Performance

Because rigorous system verification relies on accurate failure validation, we evaluate how effectively the LLM Council modules distinguish true system-level faults (e.g., mechanical or electrical failures) using natural-language requirements. To quantify this capability, we define the Fault Validation Rate (FVR) as a seed-level success metric. Based on the full distribution of experimental runs, the FVR denotes the percentage of seeds in which the Council successfully validated the failure scenario.

Dataset	$\text{Accuracy}_{\text{initial}}$	$\text{Accuracy}_{\text{tuned}}$	Improvement (%)
Dataset 1 (Hovering)	0.954	1.000	+4.82%
Dataset 2 (Lawnmower)	0.994	0.997	+0.30%
Dataset 3 (Complex)	0.688	0.847	+23.11%

Table 1: Council Loop Accuracy before and after the Adaptation Pipeline.

We evaluated and tested across 75 independent seeds. Each seed includes 409 test points with sensor drift bias and environmental noise, which may make fault validation challenging. Specifically, this evaluation tests whether the council can validate (i) the detected anomalies in environmental noise that violate conformal bounds as nuisance failures. (ii) the detected anomalies by dynamic maneuvering during normal operation. (iii) true faults with requirement-based reasoning.

The AIVV Council achieved an outstanding FVR of 100% for the Hovering Maneuver, 89.33% for the Lawnmower Mapping Pattern, and 93.33% for the Complex Mission. These results mitigate the vulnerability of the pure mathematical model and guarantee the reliability for system verification and redesign.

5.2 Adaptation Performance

In this section, we evaluate the performance of mathematical model adaptation with general anomaly detection metrics. Since we use a fundamental mathematical model, MC dropout LSTM with conformal calibration, we statistically extracted True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN) to compare the accuracy of the fine-tuned model with the initial mathematical model. Accuracy metrics can be described as follows:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{3}$$

In section 4.1, we demonstrated our total data points of the test term and described the exact fault injection point in section 4.2. Thus, we set the denominator of the metrics to 409 and compute the average accuracy based on experimental results from 75 seeds. Evaluated accuracy is demonstrated in Table 1.

In Table 1, Dataset 3 shows significant performance improvement compared to Dataset 1 and Dataset 2. Since Datasets 1 and 2 show repeated simple patterns throughout the entire time series, the mathematical model has sufficient capacity for prediction. However, for Dataset 3, which shows a complex and realistic mission pattern, adaptation shows dramatic improvement **23.11%**. With this result, we emphasize mathematical model requires sufficient fine-tuning for trustworthy V&V before mission deployment.

5.3 Ablation Study

Because the AIVV framework fundamentally verifies and validates anomaly detection monitors at the system level and is designed to overcome the contextual blindness and false-positive susceptibility of pure mathematical detectors, direct latency comparisons

Method	Dataset 1	Dataset 2	Dataset 3
Math Engine + Sentry (Baseline)	45.33%	0%	0%
+ The Council	98.67%	80%	73.33%
+ Adaptation (AIVV)	100%	89.33%	93.33%

Table 2: Failure Validation Rate (FVR) across framework phase.

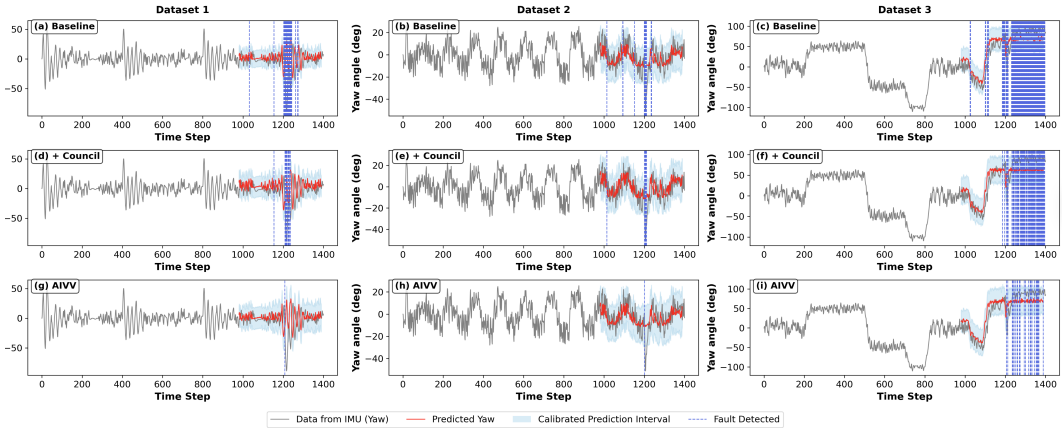


Figure 3: Ablation study comparing three framework stages (rows) across three test scenarios (columns). The mathematical baseline exhibits a high false-positive rate (FPR), which is visibly reduced when the LLM council is introduced. The full AIVV framework integrates the adaptation pipeline to achieve optimal validation.

with purely mathematical classifiers are structurally misaligned. Instead, we conduct a progressive ablation study to isolate the necessity of our architecture in all cases.

Figure 3 and Table 2 show that pure mathematical gating results in a high false positive rate (FPR), showing a lack of ability to filter out noise or dynamic maneuver-induced anomaly flags in Datasets 2 and 3. By adding the LLM Council, which achieves a Failure Validation Rate (FVR) of at least 70%, the final inclusion of the Adaptation Pipeline suppresses FPR and enables an FVR of at least 89.33%, achieving trustworthy validation across all evaluated conditions.

5.4 Role-Specific Model Alignment

To evaluate the impact of employing heterogeneous LLM agents, Table 3 shows the cyclically shifted model assignments across AIVV roles. While the optimal configuration achieved 100% Failure Validation Rate (FVR), arbitrary shifts caused performance to plummet (44.0%–56.0%) and triggered frequent JSON generation failure. For instance, Config 2 maintained 93.33% FVR by retaining a GPT-OSS model for the Failure Manager, leveraging its superior logical reasoning for trajectory analysis.

Conversely, assigning high-capacity synthesis tasks to smaller models consistently led to cognitive overload and structural errors. These results underscore the necessity of matching specific model strengths to cognitive tasks rather than relying on homogeneous or purely parameter-scaled deployments. For a detailed analysis of the resulting computational efficiency, system latency, and token consumption, refer to Appendix G.

Configuration	Deliberative Council			Adaptation Pipeline		Validation
	Req. Eng.	Fail. Mgr.	Sys. Eng.	Inspector	Tuner	FVR
Optimal	LLaMA-17B	GPT-120B	LLaMA-70B	Qwen-32B	GPT-20B	**100%**
Config 1	GPT-20B	LLaMA-17B	GPT-120B	LLaMA-70B	Qwen-32B	56.0%
Config 2	Qwen-32B	GPT-20B	LLaMA-17B	GPT-120B	LLaMA-70B	93.33%
Config 3	LLaMA-70B	Qwen-32B	GPT-20B	LLaMA-17B	GPT-120B	44.0%
Config 4	GPT-120B	LLaMA-70B	Qwen-32B	GPT-20B	LLaMA-17B	48.0%

Table 3: Systematic cyclic shift of LLM assignments across AIVV phases.

6 Conclusion

The AIVV architecture provides a hybrid neuro-symbolic approach to autonomous V&V for trustworthy control system design. By anchoring multi-agent LLM reasoning with calibrated conformal bounds, AIVV successfully digitizes the verification and validation inherent in the Human-in-the-Loop paradigm. It eliminates false positive alerts while ensuring accurate fault validation. Furthermore, the framework's clone-and-promote methodology allows for safe, dynamic model recalibration in the loop.

Future work will investigate closed-loop execution of the System Engineer's gain-tuning proposals within the inner control loop, enabling fully autonomous fault-tolerant system redesign without human intervention.

Acknowledgments

We would like to thank the support of National Science Foundation (DMS-2533878, DMS-2053746, DMS-2134209, ECCS-2328241, CBET-2347401 and OAC-2311848), and U.S. Department of Energy (DOE) Office of Science Advanced Scientific Computing Research program under the "Uncertainty Quantification for Multifidelity Operator Learning (MOLUcQ)" project (Project No. 81739), DE-SC0023161, the SciDAC LEADS Institute, and DOE–Fusion Energy Science, under grant number: DE-SC0024583.

References

Anastasios N Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

Rina Foygel Barber, Emmanuel J. Candes, Aaditya Ramdas, and Ryan J. Tibshirani. Conformal prediction beyond exchangeability. *The Annals of Statistics*, 51(2), 2023.

Yongqiang Chen, Gang Niu, James Cheng, Bo Han, and Masashi Sugiyama. Towards scalable oversight with collaborative multi-agent debate in error detection, 2025. URL https://arxiv.org/abs/2510.20963.

Mary L Cummings. Man versus machine or man+ machine? *IEEE Intelligent Systems*, 29(5): 62–69, 2014.

Cybergalactic. Marine systems simulator (MSS) simulink demos. https://github.com/cybergalactic/MSS, 2023. Accessed: 2023-10-25.

Lingyan Dong and Yan Huo. Research on fault diagnosis method for autonomous underwater vehicles based on improved lstm under data missing conditions. *Applied Sciences*, 15 (21):11570, 2025. doi: 10.3390/app152111570.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023. URL https://arxiv.org/abs/2305.14325.

Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In Maria Florina Balcan and Kilian Q. Weinberger (eds.), *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pp. 1050–1059, New York, New York, USA, 20–22 Jun 2016. PMLR. URL https://proceedings.mlr.press/v48/gal16.html.

Kyle Hundman, Valentino Constantinou, Christopher Laporte, Ian Colwell, and Tom Soderstrom. Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 387–395, 2018.

Ujin Jeon, Christopher Hixenbaugh, Michael Filimon, Benjamin Pelletier, and Vijay Gupta. Early fault detection and identification in uuvs using dropout lstm and cnn. In *OCEANS 2025 - Great Lakes*, pp. 1–7, 2025. doi: 10.23919/OCEANS59106.2025.11245166.

Ziwei Ji et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 2023.

Roham Koohestani. Agentguard: Runtime verification of ai agents, 2025. URL https://arxiv.org/abs/2509.23864.

J. C. R. Licklider. Man-computer symbiosis. *IRE Transactions on Human Factors in Electronics*, HFE-1(1):4–11, 1960. doi: 10.1109/THFE2.1960.4503259.

Natasha Chrissane Lobo, Himani H. Poojary, Lubna Katapady, Prerana Rao Adyapady, Arockiaraj Simiyon, and Thirunavukkarasu Indiran. Online sensor fault detection using machine learning algorithms on a laboratory-scale batch reactor: Lstm approach. *ACS Omega*, 11(7):12038–12051, 2026. doi: 10.1021/acsomega.5c11180. URL https://doi.org/10.1021/acsomega.5c11180.

Pankaj Malhotra, Lovekesh Vig, Gautam Shroff, and Puneet Agarwal. Long short term memory networks for anomaly detection in time series. In *Proceedings of the 23rd European Symposium on Artificial Neural Networks (ESANN)*, pp. 89, 2015.

Polster Kader Mbakob Ndagoua and Slaoui Fouad Hasnaoui. Real-time detection of short-circuit faults in power systems using lstm autoencoder. *E3S Web of Conferences*, 680:00054, 2025.

Guansong Pang, Chunhua Shen, Longbing Cao, and Anton Van Den Hengel. Deep learning for anomaly detection: A review. *ACM Computing Surveys (CSUR)*, 54(2):1–38, 2021.

Trisanth Srinivasan, Santosh Patapati, Himani Musku, Idhant Gode, Aditya Arora, Samvit Bhattacharya, Abubakr Nazriev, Sanika Hirave, Zaryab Kanjiani, and Srinjoy Ghose. Duracps: A multi-role orchestrator for dependability assurance in llm-enabled cyber-physical systems. In *2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*, pp. 63–70, 2025. doi: 10.1109/DSN-W65791.2025.00040.

The MathWorks, Inc. *IMU Sensor Fusion with Simulink*. The MathWorks, Inc., Natick, MA, USA, 2023. URL https://www.mathworks.com/help/fusion/ug/imu-sensor-fusion-with-simulink.html. Accessed: 2023-10-25.

Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O'Sullivan, and Hoang D. Nguyen. Multi-agent collaboration mechanisms: A survey of llms, 2025. URL https://arxiv.org/abs/2501.06322.

Shreshth Tuli, Giuliano Casale, and Nicholas R Jennings. Tranad: deep transformer networks for anomaly detection in multivariate time series data. *Proceedings of the VLDB Endowment*, 15(6):1201–1214, 2022.

Stian Van de Merwe, Steven C Mallam, and Salman Nazir. Trust in automation (tia): Empirical findings in supervisory control of maritime autonomous surface ships. *International Journal of Human–Computer Interaction*, pp. 1–22, 2024.

Paul Wiessner, Grigor Bezirganyan, Sana Sellami, Richard Chbeir, and Hans-Joachim Bungartz. Uncertainty-aware time series anomaly detection. *Future Internet*, 16(11), 2024. ISSN 1999-5903. doi: 10.3390/fi16110403. URL https://www.mdpi.com/1999-5903/16/11/403.

Zhiheng Xi et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.

Fabio Massimo Zanzotto. Human-in-the-loop artificial intelligence. *Journal of Artificial Intelligence Research*, 64:243–252, 2019.

A Mathematical Layer Algorithm

This appendix details the formal mathematical definitions and the offline calibration procedure for the Mathematical Engine Layer referenced in Section 3. The foundational anomaly detection method is generated using a Monte Carlo (MC) Dropout LSTM, denoted as f_θ. To quantify epistemic uncertainty at inference time without altering the network architecture, dropout is applied to both the internal LSTM layers and the output projection layer during stochastic forward passes. For a given input sample x_t, with k iteration index, the model performs N forward passes to compute the predictive mean μ_t and the predictive standard deviation σ_t (which serves as the empirical uncertainty):

$$\mu_t = \frac{1}{N}\sum_{k=1}^{N} \hat{y}_t^{(k)}$$

$$\sigma_t = \sqrt{\frac{1}{N}\sum_{k=1}^{N}\left(\hat{y}_t^{(k)} - \mu_t\right)^2}$$

To translate these point predictions into statistically guaranteed boundaries, we apply conformal prediction. Algorithm 2 outlines the complete offline procedure for training the model and establishing the conformal bound C_α alongside the uncertainty threshold τ_α using a held-out calibration set.

Algorithm 2 Mathematical Engine Layer: Training and Conformal Calibration

Require: Preprocessed training data $\mathcal{D}_{\text{train}}$, MC samples N, significance level α, calibration ratio γ

Ensure: Trained MC Dropout LSTM f_θ, conformal bound C_α, uncertainty threshold τ_α

1: Split $\mathcal{D}_{\text{train}}$ into fit set $\mathcal{D}_{\text{fit}}$ and calibration set $\mathcal{D}_{\text{cal}}$ based on ratio γ
2: Train MC Dropout LSTM f_θ on $\mathcal{D}_{\text{fit}}$ using MSE loss
3: **for** each calibration sample $x_t \in \mathcal{D}_{\text{cal}}$ **do**
4: Perform N stochastic forward passes with dropout active: $\{\hat{y}_t^{(k)}\}_{k=1}^{N} \leftarrow f_\theta(x_t)$
5: Compute predictive mean μ_t and epistemic uncertainty σ_i
6: Calculate absolute residual: $e_t = |\mu_t - y_t|$
7: **end for**
8: Construct calibration sets $\mathcal{R} = \{r_i\}$ and $\Sigma = \{\sigma_t\}$
9: Compute finite-sample corrected quantile level:

$$q = \min\left(1,\ \frac{\lceil (|\mathcal{D}_{\text{cal}}|+1)(1-\alpha)\rceil}{|\mathcal{D}_{\text{cal}}|}\right)$$

10: Calculate bounds: $C_\alpha = \text{Quantile}(\mathcal{R}, q), \quad \tau_\alpha = \text{Quantile}(\Sigma, q)$
11: **return** $f_\theta, C_\alpha, \tau_\alpha$

B Mathematical Layer Model Configuration

Data Processing and Feature: The telemetry data is processed via a sliding-window approach before being fed into the network. After applying a 70/30 split for training and testing on the 1400 total raw time-series points, the network is configured with an input dimension $d = 1$, focusing specifically on the yaw-angle dynamics of the REMUS 100.

Recurrent Neural Network: The neural network architecture comprises a multi-layered MC dropout LSTM followed by a multi-layered perceptron (MLP) head. We implemented two LSTM layers and a single dense output layer with a hidden state size of 32 dimensions. For epistemic uncertainty estimation, the MC dropout method is applied with a probability rate of $p = 0.2$ during both training and inference. During inference, the network samples $N = 30$ Monte Carlo forward passes for each LSTM layer. The mean of these $N = 30$ passes is taken as the predicted observation, while the standard deviation is extracted as the structural uncertainty of the LSTM.

Training and Calibration: The network is optimized using the Adam optimizer with a Mean Squared Error (MSE) loss computation. Prior to online inference, the LSTM generates its initial confidence bounds by constructing a baseline for normal UUV dynamics during training. The baseline significance level for the conformal prediction is set to $\alpha = 0.05$ to ensure a 95% confidence interval for normal operation uncertainty boundaries before any agent-driven dynamic adaptation occurs.

C Prompt Definitions

This section details the functional logic and requirements for each of the five LLM agents. To ensure readability within this manuscript, we present distilled versions of the prompts that highlight the unique V&V logic, voting criteria, and technical responsibilities of each role. To ensure consistent JSON formatting across all agents, we unified the instructions into a Common Formatting Block (CFB), as detailed in Section C.1. For reproducibility, the full, unedited prompt strings will be provided in our supplementary material and code repository.

C.1 Common Formatting Block (CFB)

All agents are strictly instructed to follow these constraints:

- **Response Format:** Respond ONLY with a valid, raw JSON object. Do NOT include introductory text, concluding remarks, or markdown code blocks (e.g., no ```` ```json ````).
- **Constraint:** Do not output reasoning outside the JSON structure.
- **Data Types:** The `confidence` and `alpha` fields must be numeric decimals (floats).

C.2 Prompt for each LLM agent

Failure Manager

Role: System Validation in FAILURE MODE.
Core Task: "Given that a fault may be present, is the system's response within failure management requirements?"
Responsibilities:

1. **Failure Effect:** Does max deviation exceed limits in requirements?
2. **Recovery/Damping:** Is `frame_values` converging or diverging?
3. **Baseline Context:** Compare current deviation against `frame_baseline_summary`.

Voting Logic:

- **FAIL:** True data is outside bounds with significant gradient change OR predicted data is significantly outside training bounds.
- **PASS:** True data is outside bounds, but predicted data is different; failure is contained, and the system is recovering.

[CFB Applied]
Schema: {`vote, risk_level, confidence, failure_management_assessment, reasoning`}

Requirements Engineer

Role: System Validation in NORMAL MODE.
Core Task: "Even assuming a potential fault exists, does the system's current behavior violate any operational requirement?"
Responsibilities:

1. **Normal Operation:** Verify Yaw rate range (-180 to 180) and per-step range (-10 to 10).
2. **Operational Limits:** Ensure damping stays within training envelopes.
3. **Masking Risk:** Fail if `bound_multiplier` > 2.0.

Voting Logic:

- **FAIL:** Default state if requirements are violated or predicted values shift more than noise levels.
- **PASS:** Predicted value is in per-step range despite true value being outside bounds.

[CFB Applied]
Schema: {`vote, confidence, requirement_section, reasoning, veto_reason`}

System Engineer

Role: Autopilot Gain-Tuning Expert.
Voting Logic: Use `failure_manager_findings` as the primary signal. FAIL if high uncertainty + large error or poor LSTM prediction. PASS if sudden maneuver causes drifting uncertainty.
Gain-Tuning Protocol: Adjust parameters ONLY if FM or RE vote FAIL.

- *Stale/High Uncertainty:* Increase K_p and T_d.
- *Low-Freq Oscillation:* Decrease K_p, increase T_i.
- *Divergence:* Decrease T_d, increase T_i.

[CFB Applied]
Schema: {`vote, risk_level, confidence, technical_assessment, reasoning, tuning_proposal, tuning_reasoning`}

Inspector

Role: Parameter Optimization for Tuner.
Actions:
1. **RECALIBRATE:** For transient noise (Set `new_alpha`).
2. **FINE_TUNE:** For persistent drift (Set `epochs`, `learning_rate`).
3. **TRY_BOTH:** Mixed evidence.
Constraints: $\alpha \in [0.01, 0.10]$, $epochs \in [50, 200]$, $LR \in [10^{-5}, 10^{-3}]$.
[CFB Applied]
Schema: {`majority_decision, pass_votes, fail_votes, action, new_alpha, epochs, learning_rate, reasoning`}

Tuner

Role: Conformal Prediction Analysis.
Task: Recommend operating $\alpha \in \{0.01, \ldots, 0.10\}$ based on whether new errors pass bounds at 95%, 98%, and 99%.
[CFB Applied]
Schema: {`recommended_alpha, reasoning, would_pass_at_recommended, confidence`}

D Representative Execution Traces of the AIVV Pipeline

This section provides structured JSON payloads generated by the LLM Council and Adaptation Pipeline.

council_to_inspector.json


```
{
  "timestamp": "2026-03-19T16:31:08.005585",
  "sample_id": 212,  "from_agent": "council",  "to_agent": "inspector",
  "payload": {
    "loop": 2,
    "votes": [
      {
        "agent": "req_eng",
        "vote": "PASS",        "confidence": 0.7,
        "reasoning": "The error magnitude (1.6831555366516113) is slightly above..."
      },
      {
        "agent": "fail_mgr",
        "vote": "PASS",        "confidence": 0.92,
        "reasoning": "Peak deviation 16.81\u202f<\u202f37.57, range 17.47\u202f<..."
      },
      {
        "agent": "sys_eng",
        "vote": "PASS",        "confidence": 0.9,
        "reasoning": "The failure manager findings indicate a PASS vote with the..."
      }
    ]
  }
}
```


inspector_to_tuner.json


```
{
  "timestamp": "2026-03-19T16:31:00.123319",
  "sample_id": 212,  "from_agent": "inspector",  "to_agent": "tuner",
  "payload": {
    "majority_decision": "PASS",
    "pass_votes": 3,    "fail_votes": 0,
    "vote_details": [
      {
        "agent": "req_eng",
        "vote": "PASS",        "confidence": 0.8,
        "reasoning": "The current error magnitude is 1.6453118324279785, which...",
        "risk_level": null
      },
      {
        "agent": "fail_mgr",
        "vote": "PASS",        "confidence": 0.92,
        "reasoning": "The observed peak deviation, overall range, and oscillati...",
        "risk_level": "LOW"
      },
      {
        "agent": "sys_eng",
        "vote": "PASS",        "confidence": 0.95,
        "reasoning": "The failure manager findings indicate a PASS vote with th...",
        "risk_level": "LOW"
      }
    ],
    "action": "RECALIBRATE",
    "reasoning": "The error (1.645) slightly exceeds the bound (1.445) with low...",
    "new_alpha": 0.05
  }
}
```

tuner_to_sentry.json


```
{
  "timestamp": "2026-03-19T16:31:02.631120",
  "sample_id": 212,  "from_agent": "tuner",  "to_agent": "sentry",
  "payload": {
    "new_prediction": -0.4726104736328125,
    "new_bound": 1.220279983597098,
    "new_error": 1.6831555366516113,
    "new_uncertainty": 0.12548138201236725,
    "applied_alpha": 0.1,
    "passes_reevaluation": false
  }
}
```


E Detailed Dataset Explanation

Dataset 1 (Hovering). A low-dynamic operational scenario in which the UUV attempts to maintain a fixed position and heading through small, damped corrective control actions. This case is designed to evaluate the fundamental anomaly detection capability under station-keeping conditions, where normal control-induced oscillations must be distinguished from genuine fault signatures.

Dataset 2 (Lawnmower Mapping Pattern). A highly structured, repetitive grid-mapping operation designed by dynamic, periodic turning maneuvers. This case tests the ability of the system to distinguish between aggressive control inputs and mechanical faults

Dataset 3 (Complex Mission). A mission operative maneuver involving continuous, aperiodic course corrections and varying operational speeds, designed to evaluate the advantage of calibrated uncertainty boundary and adaptive thresholding capabilities.

F Detailed Analysis of Role-Specific Model Alignment

As introduced in Section 5.4, the performance disparities across the model allocation sweep are directly related to how well each model family handles its assigned task. The optimal AIVV configuration relies on the following structural alignments:

GPT-OSS Family (Mathematical & Sequential Logic). The GPT architecture excels at sequential logical reasoning and complex trajectory deduction. This makes it uniquely suited for the Failure Manager, which must calculate post-fault divergence and track oscillation sequences. When a smaller 20B GPT model was assigned to this role (Config 2), accuracy remained highly robust (93.33%). Conversely, assigning LLaMA or Qwen to this role caused critical evaluation failures, even at the massive 70B-parameter count.

LLaMA-4-17B (Deterministic Compliance). Highly instruction-tuned models at this parameter scale are optimized for rigid, low-latency rule adherence. This perfectly matches the Requirements Engineer, which performs strict, static numerical limit checks. It executes these deterministic tasks efficiently without the computational overhead of massive context windows.

High-Capacity Models (70B+ Domain Synthesis). The System Engineer requires deep domain synthesis to evaluate LSTM vulnerability metrics alongside generating complex JSON payloads for the PID gain-tuning proposal. Only high-capacity models (`LLaMA-3.3-70B` or `GPT-OSS-120B`) possessed the representational bandwidth for this task. Relegating smaller models to this role consistently caused cognitive overload, resulting in broken JSON structures.

Scenario	Agent Role	Avg Calls / Seed	Avg Latency (s)	Avg Input Tokens	Avg Output Tokens
Dataset 1 (Hovering)	Requirements Engineer	27.16	0.54	1394.8	135.4
	Failure Manager	27.16	1.66	1188.3	493.5
	System Engineer	27.16	1.02	1954.7	242.0
	Inspector	1.25	1.69	1014.7	682.0
	Tuner	1.25	1.24	465.9	579.1
Dataset 2 (Lawnmower)	Requirements Engineer	15.29	0.52	1398.0	148.2
	Failure Manager	15.29	2.05	1190.6	549.1
	System Engineer	15.29	0.93	1974.3	216.4
	Inspector	3.11	1.68	1018.6	678.0
	Tuner	3.11	1.13	465.9	549.2
Dataset 3 (Complex)	Requirements Engineer	227.03	0.59	1403.5	157.4
	Failure Manager	227.03	1.47	1189.5	482.2
	System Engineer	227.03	0.84	1999.5	201.0
	Inspector	87.00	1.83	1012.0	749.6
	Tuner	87.00	1.30	466.4	632.3

Table 4: Average API calls, latency, and token consumption per seed.

G Analysis of Efficiency, Latency, and Token Consumption

LLM Agent Call Efficiency Sentry escalated only about 5% of the samples to the LLM council in normal cases (averaging $\sim$20 calls per seed for Datasets 1 and 2), and roughly 50% during the highly dynamic complex mission ($\sim$230 calls per seed). This demonstrates the efficiency of the mathematical bounds.

As the data was filtered for nuisance faults with ground rules based on natural language requirements in the Phase 2 LLM council, the computationally expensive Phase 3 Adaptation Pipeline was invoked an average of only 2 times per seed for the simpler maneuvers, scaling to 87 times for the complex model. During the Tuner's clone-and-verify protocol, the system achieved average successful model promotions of 1.08, 1.31, and 10.72 per seed for Datasets 1, 2, and 3, respectively. This demonstrates that triggering a fine-tuning cycle does not guarantee a live model update; promotion strictly requires the candidate model to satisfy the conformal mathematical bounds. Furthermore, the increased promotion rate in Dataset 3 illustrates that as operational dynamics become more complex, the AIVV framework safely and proportionally increases its update frequency to maintain performance.

Token Consumption and Context Stability. Input tokens remain stable ($\sim$1200–2000) across all missions because prompts rely on fixed-size sliding windows, preventing context bloat. Output tokens vary predictably by role: analytical agents produce concise JSON ($\sim$150 tokens), while synthesis agents generate up to $\sim$750 tokens for parameter adjustments.

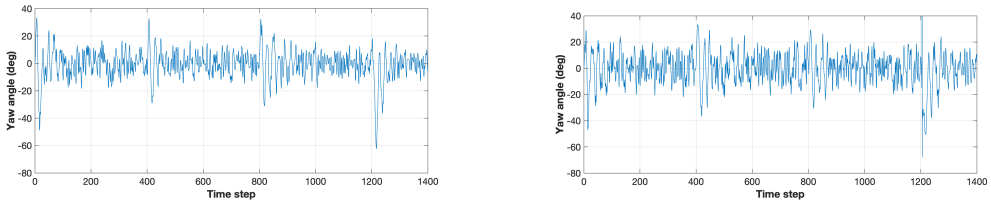
System Latency and Execution Dynamics. Individual latencies range from $\sim$0.5 to $\sim$2.0 seconds. While individual latencies are low, a full escalation through the adaptation pipeline yields a worst-case end-to-end latency of approximately 6.0 seconds per sample.

H Hardware and Inference Infrastructure

The AIVV framework was executed on the Intel Data Center GPU Max Series accelerators. This ensures high capacity and high bandwidth, with up to 408 MB of L2 cache (Rambo) based on discrete SRAM technology, 64 MB of L1 cache, and up to 128 GB of high-bandwidth memory. We used the GroqCloud API via OpenAI-compatible endpoints, leveraging Groq's custom Language Processing Units (LPUs).

I AIVV Gain-tuning results verification

In this section, we implement the gain-tuning proposal in our testbed REMUS 100 Simulink model to verify the reliability of the system engineer LLM model. Based on the V&V results from the Requirements Engineer and Failure Manager, the system engineer proposes a new gain parameter for the REMUS 100 controller. Specifically, we provided 3rd-order LP filter parameters (Max velocity, Relative damping ratio, Natural frequency) and PID controller



(a) Before gain-tuning (b) After gain-tuning

Figure 4: Effect of gain-tuning on REMUS 100 hovering (Dataset 1) yaw response

parameters (Nomoto Time Constant T, Nomoto Gain K, Proportional Gain Kp, Derivative Time Td, Integral Time Ti).

V&V summary

```
V&V Summary (Agentic System Verification & Validation)
[Requirements Engineer -- Normal Mode V&V]
  Num of agent calls : 24 | Num of FAIL  : 23
  Test result : FAIL  -- 23/24 sampled windows violated operational requirements.
  ↪  First violation at sample 221: Error magnitude exceeds max allowable error
  Violation details (up to 5):
    - Sample  221 | Operational Limits | Error magnitude exceeds max allowable error
      ...
    - Sample  225 | Operational Limits | Error exceeds max allowable error
[Failure Manager -- Failure Mode V&V]
  Num of agent calls : 24 | Num of FAIL  : 11
  Test result : FAIL  -- 11/24 fault-suspect windows violated failure management
  ↪  requirements. First violation at sample 225: peak_deviation=77.00,
  ↪  response=DIVERGING, oscillation_count=0. peak_deviation=77.00 exceeds
  ↪  max_failure_effect=72.95
  Violation details (up to 5):
    - Sample  225 | peak=77.00 | DIVERGING | osc=0 | peak_deviation=77.00 exceeds
    ↪  max_failure_effect=72.95
      ...
    - Sample  229 | peak=88.81 | DIVERGING | osc=1 | peak_deviation=88.81 exceeds
    ↪  max_failure_effect=72.95
[System Engineer -- Active Optimizer]
  Num of agent calls : 24 | Fail Votes  : 23
  Gain-Tuning Proposals (23 unique samples, triggered by FM/RE FAIL, showing up to 5):
    - Sample  221 | Triggered by: RE | SE Vote=FAIL | Params: {'Kp': 0.6, 'Ti': 19.0,
    ↪  'Td': 1.0, 'Reference_Max_Velocity': 9.5}
      Reason: Since the requirements engineer voted FAIL due to an operational limit
      ↪  violation, the tuning proposal aims to reduce the error magnitude. The
      ↪  proportional gain (Kp) is increased to
      ...
    - Sample  225 | Triggered by: FM+RE | SE Vote=FAIL | Params: {'Kp': 0.7, 'Ti':
    ↪  15.0, 'Td': 1.2, 'Reference_Max_Velocity': 9.0}
      Reason: Since both failure manager and requirements engineer voted FAIL,
      ↪  adjusted parameters are proposed. Increased Kp to 0.7 to reduce error,
      ↪  decreased Ti to 15.0 to reduce oscillations,
```

With the given system engineer's gain-tuning proposal, we implemented the last proposal (Kp: 0.5 → 0.7, Ti: 20 → 15, Td: 1.0 → 1.2, Max Velocity 10.0 → 9.0).

Figure 4 compares the yaw angle responses before and after applying the proposed parameters. The gain-tuning verification serves as a proof of concept demonstrating that AIVV can automatically translate failure-mode analysis into structured, systemically reasoned corrective action proposals. While full-control optimization remains an open engineering challenge, the systemically coherent parameter adjustments confirm that the System Engi-

neer successfully bridges the V&V gap between fault identification and actionable system redesign guidance, a capability that previously required human domain expertise.